BhumiSetu

India's Land Decision Infrastructure

Master Product Requirements Document

Parts 12 to 15: AI Architecture, Database, APIs, & Tech Stack

Document Grade: Investor & Engineering Ready

Date: July 2026

PART 12: AI ARCHITECTURE

BhumiSetu is an AI-native company. We do not use LLMs as a wrapper for chat; we use Multi-Agent systems to power deterministic workflows, extracting structured intelligence from unstructured Indian real estate chaos.

1. The Multi-Agent Ecosystem

- **Document Intelligence Agent (DIA):** Ingests scans, applies OCR, and translates vernacular records (Hindi, Marathi, Tamil) to English. Uses Vision-Language Models (VLMs) to read handwritten mutation entries.
- **Decision Agent:** The reasoning engine. It compares the DIA's structured output against the Land Decision Graph to flag anomalies (e.g., "The seller's name on the deed does not match the 7/12 extract").
- **Workflow Agent:** The dispatcher. If the Decision Agent flags a missing boundary, the Workflow Agent automatically triggers an API call to dispatch a physical surveyor via the Partner App.

2. RAG & Vector Database

We utilize **Retrieval-Augmented Generation (RAG)** to give our LLMs deep context on Indian property law.

- **Vector Database:** Pinecone / Milvus.
- **Knowledge Base:** State-specific land revenue codes, Supreme Court property judgments, and historical circle rates embedded into vector space.
- **Process:** When assessing a complex succession dispute in Maharashtra, the Decision Agent retrieves the specific clauses from the Maharashtra Land Revenue Code, 1966, before generating an alert.

3. Guardrails & Human-in-the-Loop (HITL)

AI hallucinations in legal matters are catastrophic. We employ strict programmatic guardrails:

- **Format Guardrails:** Output must perfectly conform to a predefined JSON schema. If it fails, the prompt is retried.
- **Confidence Thresholds:** If the DIA's extraction confidence drops below 92% (often due to poor handwriting), the task is immediately routed to a human data-entry operator (Exception Handler).

PART 13: DATABASE DESIGN

The system requires a robust relational database with advanced geospatial capabilities.

1. Core Infrastructure

Primary Database: PostgreSQL (hosted on Amazon RDS).
Geospatial Extension: PostGIS (critical for calculating polygon overlaps, area calculations, and spatial queries).

2. Entity Relationship Summary

Entity	Core Attributes	Relationships	Indexes
Users	id, type (retail/enterprise), kyc_status, auth_provider	HasMany(Cases, Payments)	idx_email, idx_phone
Parcels	id, state, district, survey_no, geom (Polygon), trust_score	HasMany(Documents, Cases)	GIST(geom), idx_survey_no
Cases	id, user_id, parcel_id, workflow_state, sla_deadline	BelongsTo(User, Parcel)	idx_workflow_state
Tasks	id, case_id, partner_id, task_type, geo_fenced_lat_lng, status	BelongsTo(Case, Partner)	idx_partner_status

PART 14: API DESIGN

BhumiSetu operates on an API-first approach, enabling a headless architecture where the Web, Mobile Apps, and Enterprise clients consume the exact same endpoints.

1. Architecture & Authentication

- **Style:** RESTful for Core CRUD; GraphQL for complex dashboard data fetching to prevent over-fetching.
- **Auth:** JWT (JSON Web Tokens) with short-lived access tokens and HttpOnly refresh tokens. Enterprise clients use API Keys with strict rate limiting.

2. Key Endpoints

POST /api/v1/cases/initiate

Purpose: Starts a new land diligence workflow.

```
Request Payload:
{
  "service_tier": "field_verification",
  "parcel": {
    "state": "MH",
    "district": "Pune",
    "survey_no": "142/B"
  }
}

Response (201 Created):
{
  "case_id": "cas_987xyz",
  "status": "enriching_data",
  "estimated_completion": "2026-08-05T10:00:00Z"
}
```

PART 15: TECHNOLOGY STACK

Optimized for rapid iteration, high scalability, and heavy AI/ML workloads.

1. Application Layer

- **Frontend (Web):** Next.js (React), Tailwind CSS, Framer Motion, Vercel hosting.
- **Frontend (Mobile):** React Native (Expo) - unified codebase for Customer and Partner apps.
- **Backend (Core API):** Node.js (NestJS) written in TypeScript for strict typing and scalable microservices architecture.
- **Backend (AI/Data):** Python (FastAPI) for deploying custom agents, interacting with LLMs, and running geospatial algorithms.

2. Infrastructure & DevOps

- **Cloud Provider:** AWS (Amazon Web Services).
- **Compute:** Amazon EKS (Elastic Kubernetes Service) for container orchestration, allowing independent scaling of the Node.js API and the Python AI workers.
- **Storage:** Amazon S3 for document storage (PDFs, Images).
- **CI/CD:** GitHub Actions for automated testing and zero-downtime deployments.
- **Infrastructure as Code (IaC):** Terraform.

3. Security & Monitoring

- **Monitoring:** Datadog (APM, Logs, Infrastructure).
- **Error Tracking:** Sentry.
- **Data Security:** AES-256 encryption at rest (RDS/S3). TLS 1.3 in transit. Strict IAM roles and VPC peering.